


Powered by Arizona State University®

**Ing. En Software y Mineria de Datos**

**1 semestre**

**Sistema de seguimiento de tareas académicas- DISEÑO**

**Maestro: Vidkar Anibal Delgado Gallardo**

**Alumno: Leonardo Hernandez Ramirez**

**Aisha Quijada Meza**

**Aturo Ochoa**

**Carlos Bolaños**

# Propósito del documento

En este documento se especifica el diseño que tendrá el sistema para registrar las tareas de los estudiantes. Se especifican la estructura que tendrá el sistema, se detalla el funcionamiento de los modulos y sus relaciones con las las funciones que están siendo usadas dentro del código.

El sistema que se define como un software para la administración de entrega de tareas, este se compone de modulos donde son dividididaas las funciones que integrarán al programa. Entre estas se ecnuentran: Añadir tareas (asignarles fecha, descripción y materia), marcar como completadas las tareas, añadir recordarios, mandar alertas donde se recuerde al usuario de la entrega de tareas, filtrar materias y tareas con un buscador.

El diseño del sistema se impletentará en C, la estructura se divivide en distintos modulos donde se especifican las distintas funciones (como registrar una tarea incluyendo sus datos), y la forma en la que el sistema funciona es a través de la consola del compilador.

Se cumplen los requisitos que se detallaron en la declaración del software, como registrar los datos del software, cambiar fechas, ajustar automaticamente la prioridad, tener un buscador, completar tareas. Tambien se respetan las restricciones planteadas previamente y se sigue la trazabilidad usando los requerimiento y los modulos donde se cumple la función que se planteó.

El sistema restringe el uso de cualquier implementación de cualquier programa externo, esto significa no poder conectarse con algún servicio y por ende ninguna API será requerida. Como este programa no involucra ningún modulo que conecte con internet o alguna comunicación que requiera mantenimiento constante, no requerirá versiones posteriores (a menos de que exista un cambio importante del programa) y es por esto que el usuario tendrá el rol de administrador. Esta será la manera en la que el sistema funcione.

Arquitectura del software

Por decisión del equipo, el sistema de seguimiento de tareas académicas se desarrollará en lenguaje C, optando por una interfaz de usuario basada en la consola.

Para lograrlo, se utilizará la siguiente estructura:

Funciones principales: Desplegar opciones enumeradas, capturar los datos ingresados por el usuario mediante el teclado. Ejecutar el buscador para filtrar tareas, calcular el tiempo restante comparando fechas, reordenar la lista según la prioridad.

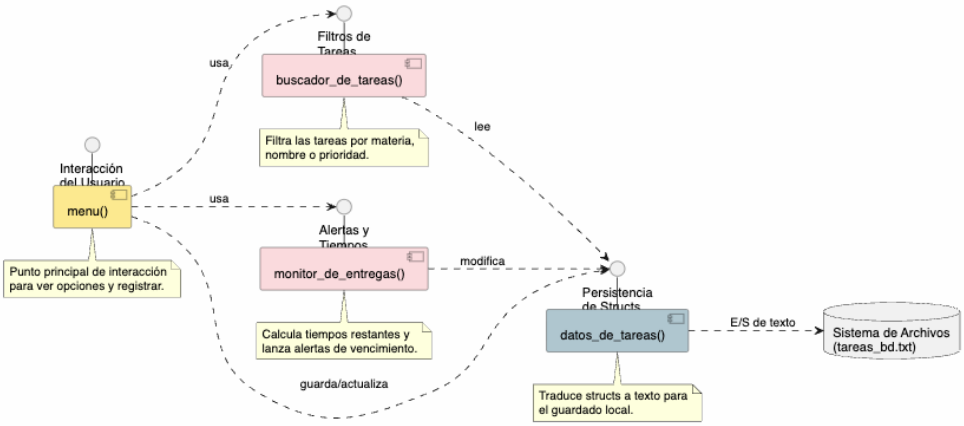
Archivos del ordenador: Dado que el sistema es de uso personal y no requiere conexión a internet ni bases de datos en red, esta capa garantiza que la información sea consisa y perdurable entre las sesiones del usuario.

Módulo buscador_de_tareas(): Encargado de ejecutar el buscador para filtrar tareas.

Módulo monitor_de_entregas(): Se encarga de calcular el tiempo restante comparando fechas, reordenar la lista según la prioridad o proximidad de entrega, y cambiar el estatus de las actividades (ejemplo: de pendiente a ¡completada!).

Módulo datos_de_tareas(): Dado que el sistema es de uso personal y no requiere conexión a internet ni bases de datos en red, esta capa garantiza a través de este módulo que la información sea concisa y perdurable entre las sesiones del usuario.

Diagrama



DISEÑO DE INTERFACES

La navegación se realizará mediante menús numerados.

Menú Principal:

```
========================================
      GESTOR DE TAREAS ACADÉMICAS
========================================
1. Registrar nueva tarea
2. Ver lista de tareas
3. Buscar / Filtrar tareas
4. Marcar tarea como completada
5. Editar fecha de entrega
6. Salir
========================================
Elija una opción: _
```

Si el usuario elige la opción número 2, el programa mostrara un segundo menú (submenú) con las siguientes características:

```
--- VISUALIZACIÓN DE TAREAS ---
1. Ver todas (Pendientes y Completadas)
2. Ver solo tareas Pendientes
3. Ver tareas Vencidas
4. Regresar al menú principal
```

## Descripción de las entradas que proporcionará el usuario

El usuario interactuará con el sistema exclusivamente a través del teclado.

Las entradas principales serán:

Comandos de navegación: Números enteros (1, 2, 3, etc.) para seleccionar las opciones de los menús.

Datos de registro de tarea:

Nombre de la tarea: Cadena de texto (ejemplo: Reporte de laboratorio).

Materia: Cadena de texto (ejemplo: Física General).

Fecha límite: Formato numérico estructurado, como DD/MM/AAAA (ejemplo: 15/11/2026).

Prioridad: Selección numérica (ej. 1 para Alta, 2 para Media, 3 para Baja).

Parámetros de búsqueda: Cadenas de texto o números dependiendo del filtro elegido.

Confirmaciones de acción: Caracteres simples como s o n para confirmar el marcado de una tarea o salir del sistema.

## Descripción de las salidas, mensajes y resultados (Outputs)

El sistema responderá a las acciones del usuario limpiando la pantalla y mostrando mensajes claros:

Listados claros y faciles de entender: Los resultados de las tareas se imprimirán en formato de tabla simulada con texto plano, mostrando: (Materia, Tarea, Fecha, Prioridad , Estado).

Mensajes de éxito: mensaje inmediato tras una acción correcta.

Ejemplo: Tarea registrada correctamente en el sistema.

Ejemplo: La tarea “Proyecto Final” ha sido marcada como completada.

Alertas de vencimiento:

Al entrar al menú o visualizar la lista, el sistema imprimirá advertencias para las tareas próximas a vencer.

Ejemplo: ALERTA: Tienes 1 tarea que vence mañana.

Mensajes de error y validación:

Avisos cuando el usuario ingresa un dato incorrecto o que no concuerda con el formato del menú en el programa.

Ejemplo: Opción no válida. Por favor, ingrese un número del 1 al 4.

Ejemplo: Formato de fecha incorrecto. Intente con Dia/Mes/Año.

## Diseño de módulos y funciones

Menu.h;

Objetivo: Desplegar las opciones principales del sistema en la consola.

Parametros de entrada: Ninguno (Aceptará enteros cuando el usuario seleccione una opción).

Valor de retorno: nada, será void

Descripción: Imprime el menú con formato, listando opciones como 1. Agregar tarea, 2. Buscar, 3. Ver próximas, 4. Salir.

Datos_de_tareas.h

Objetivo: Guardar una nueva tarea en el sistema.

Parametros de entrada: Se guardarán en una cadena de char el nombre, un arreglo de numeros para comparar si es mayor

Valor de retorno: int (1 si se encontró y actualizó, 0 si no existe el ID).

Descripción: Añade un nuevo registro al final del arreglo y aumenta el contador de tareas totales.

Buscador_de_tareas.h

Objetivo: Mostrar todas las tareas asociadas a una asignatura específica.

Parametros de entrada: Cadena de texto con la materia, arreglo de tareas y total de tareas.

Valor de retorno: El resultado de la busqueda, si es verdadero se imprimiran los resultados con un for y un if, pero si no se pone con el resultado del else.

Descripción: Itera sobre el arreglo de estructuras, usa for e if para buscar lo que los resultados que estén en el arreglo y satisfagan el resultado, pero en caso de no haber nada mostrará un mensaje en el cual se dice que no hay nada.

Monitor_de_tareas.h

Objetivo: Listar las tareas que vencen pronto.

Parametros de entrada: Arreglo, total de tareas y un entero con el umbral de días (se usará una estructura con if).

Valor de retorno: Mostrará una lista de prioridades, y sabrá que mostrar ya que comparará fechas hasta llegar a la mas pronta.

Descripción: Compara la fecha de vencimiento de cada tarea pendiente con la fecha actual. Si la diferencia es menor o igual a dias_limite, la imprime en pantalla con una alerta visual.

Diseño de datos

El sistema usará dos tipos de datos: enteros y char. Usará estos dos ya que solo son lo unicos que serán utiles para el programa, tambien es importante señalar que se acomodarán en estructuras (struct) para poder organizar la entrada de estos.

Para el apartado de la fecha se usarán 3 variables de enteros para día, mes y año. Para el apartado de las tareas se usará uno para asignarle un ID a la tarea (este se incrementa automaticamente), 3 cadenas para el titulo, materia y descripción.

El usuario al tener un unico rol tambien podrá administrar sus tareas, acceder a la consulta de sus tareas y saber todo tipo de dato relacionado con lo que este haya introducido.

```
//aqui van los datos de la fecha xd
 struct Fecha{
    int dia;
    int mes;
    int año;
} ;

//aca van las tareas
 struct Tarea {
    int id_tarea;
    char materia[30];
    char descripcion[150];
    Fecha fecha_entrega;
    int estado;
};
```

Diagramas de diseño

Diagrama de flujo para la función agregar_tarea

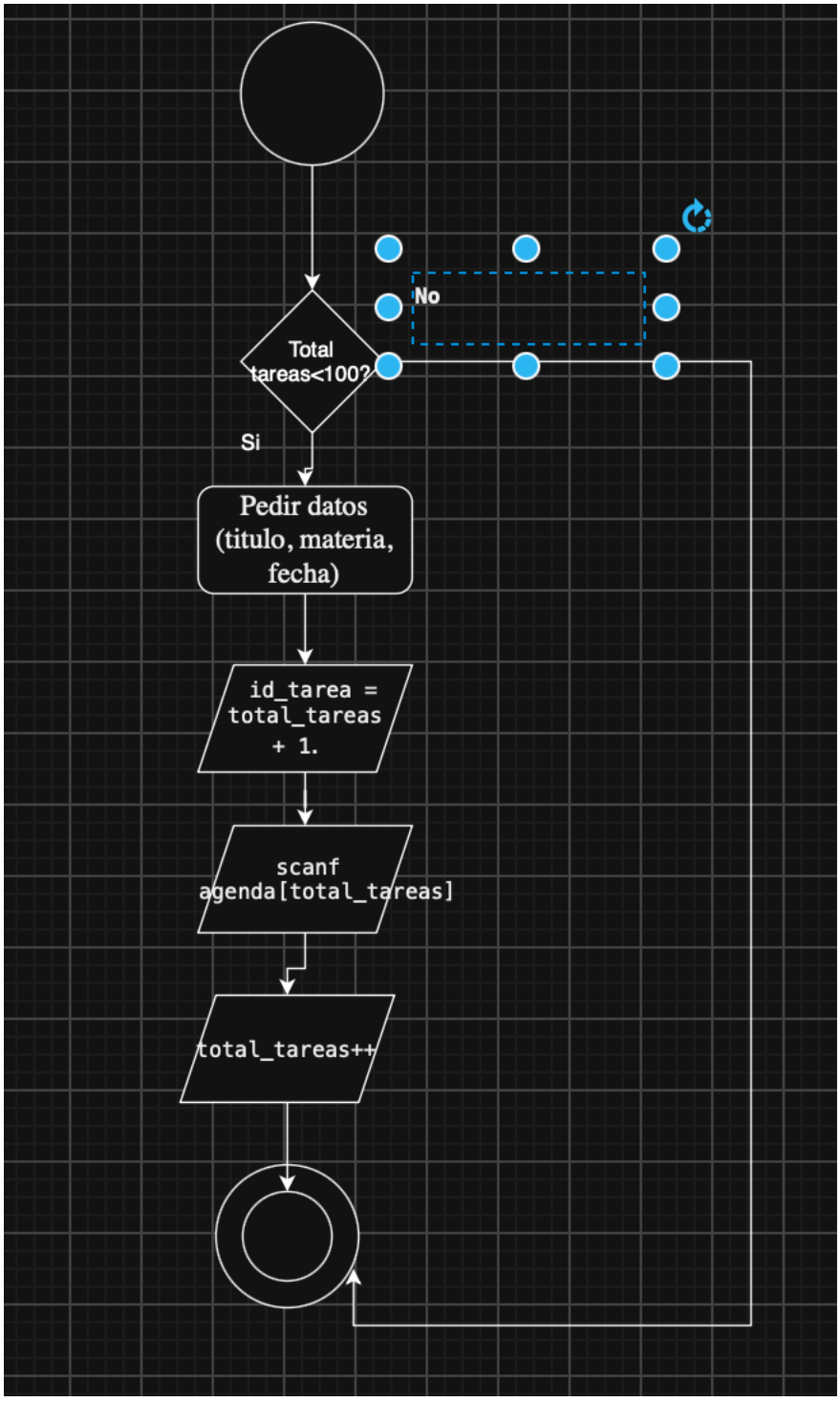


Diagrama de casos de uso

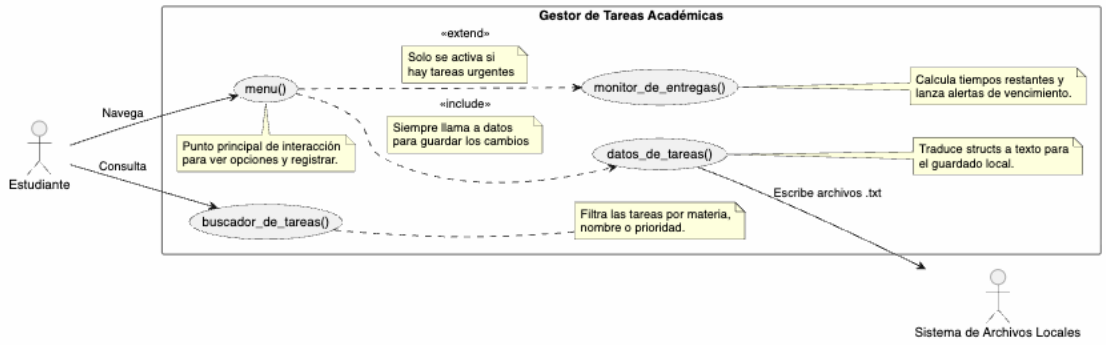



Diagrama de secuencia

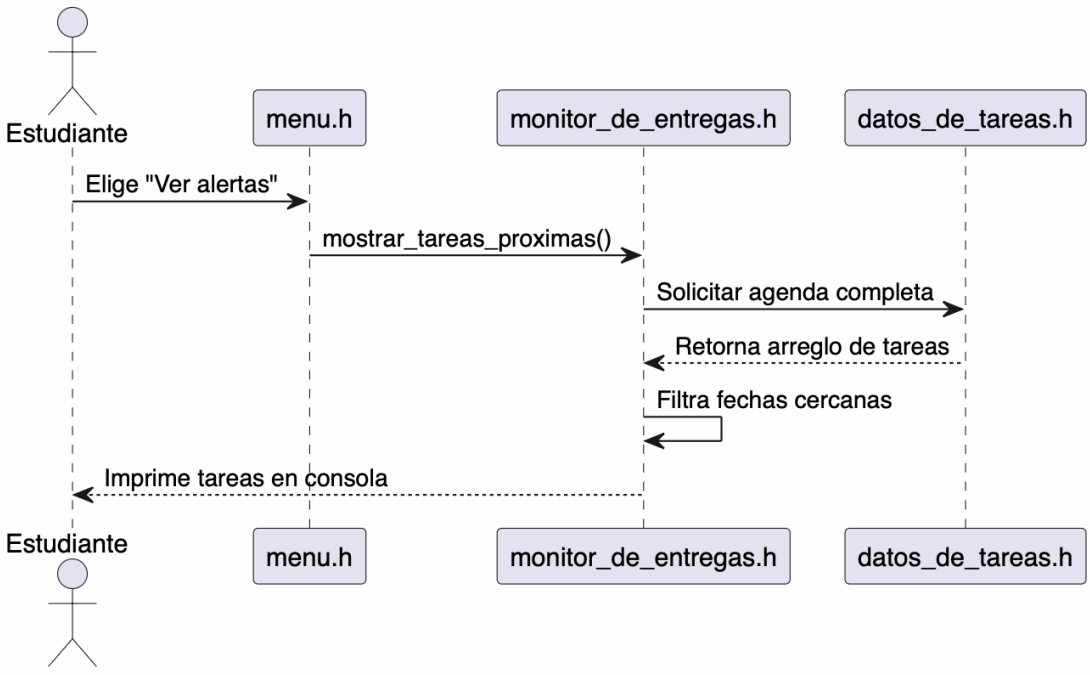
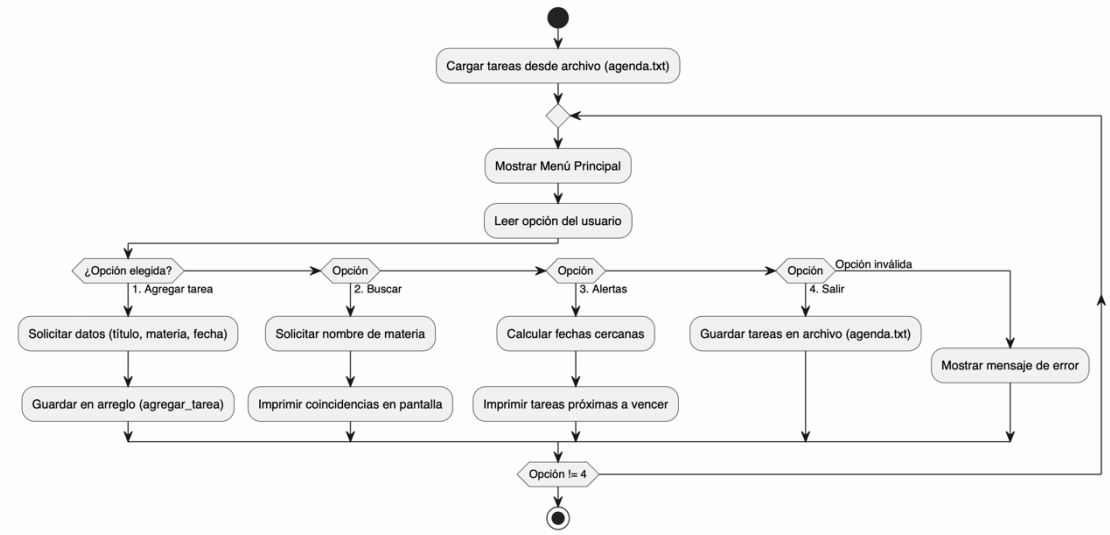


Diagrama de actividades



Trazabilidad

| ID del Requerimiento | Descripción | Modulo | Caso de prueba |
|---|---|---|---|
| RF 001 | Formulario ágil e ingreso de datos facilitado | Datos_de_tareas() | CP 01 |
| RF 002 | Selector o campo de fecha destacado en la interfaz | Monitor_de_entregas() | CP 02 |
| RF 003 | Cambiar automaticamente la fecha de entrega | Monitor_de:entregas() | CP 03 |
| RF 004 | Tener un buscador | Buscador_de_tareas() | CP 04 |
| RF 005 | Marcar como completada las tareas | Datos_de_tareas() | CP 05 |
| RF 006 | Enviar alertas cuando se acerque la fecha de entrega | Monitor_de_tareas() | CP 06 |
| RNF 001 | Guardar una tarea en menos de 2 segundos | Datos_de_tarea() | CP 07 |
| RFN 002 | Ser una interfaz intuitiva | Menu() | CP 08 |